

AI, coding, and judgment

Coding Lesson 1 · Math Camp 2026

Rony Rodriguez-Ramirez

Harvard Kennedy School · API 209

Thursday, August 13, 2026

Lesson map

Welcome and the question

Where we begin

Why coding still matters

From a model to an agent

Learning with an agent

Delegating a bounded task

From Lesson 1 to Lab 1

Welcome



Rony Rodriguez-Ramirez

- ▶ I work in education policy, with a focus on economics of education.
- ▶ I have worked with research teams at the World Bank and Innovations for Poverty Action.
- ▶ I like coding when it helps us ask a clearer question.
- ▶ I will teach these coding lessons and work with you during the labs.

We have different starting points

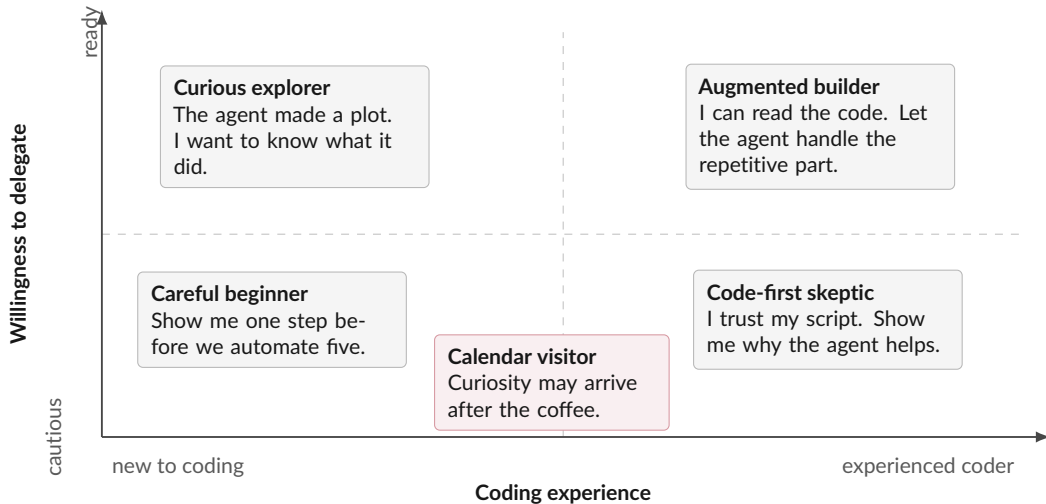
Some people in this room are writing code for the first time. Others already use R for research. Many people know ChatGPT. A coding agent that works inside a project will be a newer experience for most of us.

This is fine

We will use one common project. Each student can take steps that fit their experience with coding and their current comfort with AI.

Your starting point can also change from one task to another.

Some AI + coding archetypes



These positions are approximate. Your position can change with the task.

The question for today

What should we delegate, to which system,
and with what evidence?

The task and the evidence determine which tool belongs in the workflow.

What I want you to do by the end

1. Distinguish a model, a chat interface, a coding agent, and a harness.
2. Explain the different jobs of R, RStudio, Quarto, and Codex.
3. Decide whether a task should stay with you or can be delegated.
4. Write one bounded request with a visible completion check.
5. Inspect an agent result before you keep it.

Today we need enough shared understanding to use this workflow together. Expertise will grow through practice during Math Camp and the semester.

Is it still worth learning to code?

Yes, because code helps us

- ▶ make an instruction precise;
- ▶ repeat the same operation;
- ▶ inspect what a tool produced;
- ▶ change one part and test it.

Learning code is less about remembering every function. It is more about reading, predicting, changing, testing, and explaining.

Andrew Ng argues that a lower barrier to coding increases what people can build. Even a small amount of coding knowledge can therefore support more ambitious work (Ng, 2023, 2025).

The target has changed

A useful coding learner can:

- ▶ read a proposed script and say what it should do;
- ▶ change the relevant line without changing the question;
- ▶ run a small test and interpret the result;
- ▶ find where a claim enters the code;
- ▶ notice when the code and the policy question separate.

Syntax still matters alongside supervision, testing, and interpretation.

Why the workflow changed

Earlier loop

1. Write code.
2. Run code.
3. Search the error.
4. Try again.

Agent-assisted loop

1. Define the result and evidence.
2. Let the agent inspect or propose.
3. Watch files, commands, output, and the diff.
4. Test, then accept, revise, or reject.

The agent makes production faster. It also makes supervision part of computational literacy.

Lesson map

Welcome and the question

From a model to an agent

The parts of the system

Choose the task first

Learning with an agent

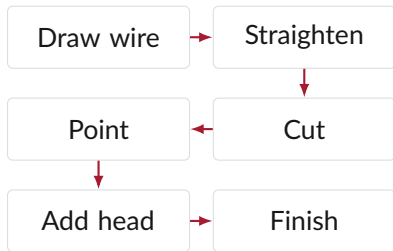
Delegating a bounded task

From Lesson 1 to Lab 1

A pin factory begins with divided work

“The greatest improvements in the productive powers of labour” grew from the division of labour.

Adapted from Smith (1776, Book I, Chapter I).



Specialization raises a coordination question: who defines the task, passes context, checks the handoff, and accepts the final product?

AI changes the division of analytical work



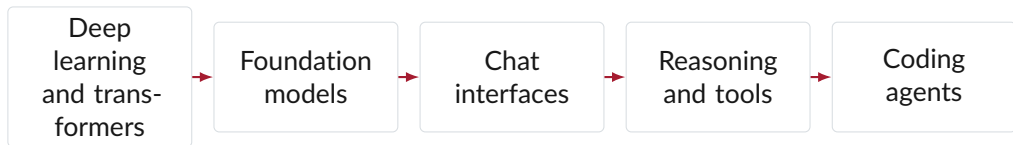
The productivity gain depends on coordination. A useful workflow makes the task, context, permissions, output, and acceptance check visible.

AI includes several kinds of systems

Term		Working definition for this course
Artificial intelligence	intelli-	A broad family of systems for prediction, perception, language, planning, or action.
Large language model	language	A model that produces language, code, or a tool request from an instruction and context.
Chat interface		A message window around a model.
Coding agent		A model working in a loop with tools that can act inside a project.

We use these terms separately because they imply different actions and different things to verify.

How we arrived at coding agents



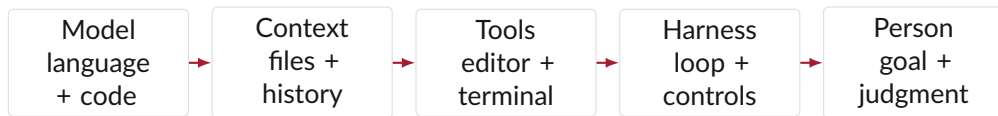
The shift from a simple chatbot to a project agent happened in stages. Each stage changed who can execute, what the system can touch, and what evidence we can observe.

From conversation to action

Experience	Who executes	What we can observe
Chat	A person copies, pastes, and runs each step.	A response.
Code completion	A person works; the model suggests nearby code.	A code fragment.
Agent	The system plans, uses tools, observes, and continues.	A sequence of actions.
Coding agent	The agent operates on project files and the terminal.	Diffs, commands, logs, and artifacts.

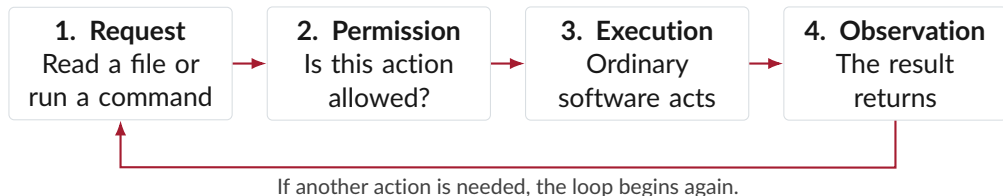
Codex and Claude Code are coding agents. Their project access allows them to read files, use tools, and return observable results (OpenAI, 2026; Anthropic, 2025).

A coding agent is a system of parts



The harness connects the model to the environment and applies permissions. We still define the objective, the standard, and the final decision.

Watch one tool-call loop



This loop helps us ask what actually happened. Evidence of an action comes from the exposed tool call, result, or log.

Four course tools, four jobs

Tool	What it does	Decision left to the analyst
R	Runs data and statistical operations.	Whether an operation represents the intended concept.
RStudio	Keeps scripts, console, files, objects, and plots visible.	Whether the code is substantively correct.
Quarto	Combines code, prose, and output in an optional notebook.	Whether the narrative follows from the evidence.
Codex	Reads, proposes, edits, runs, and checks inside a project.	Whether its output deserves our trust.

Choose the tool after the task

Need	Smallest useful system	Acceptance evidence
Calculate a summary	R script	A hand check and a reproducible result
Explain an error	Chat or agent with the error and code	Explanation matches a small test
Modify a project	Coding agent with limited access	Diff, clean run, and relevant checks
Decide a policy interpretation	Person with evidence	Definitions, assumptions, and argument

More autonomy creates more places where something can go wrong. Start with the simplest system that lets you verify the result.

Lesson map

Welcome and the question

From a model to an agent

Learning with an agent

Understanding and cognitive debt

A learning routine

Delegating a bounded task

From Lesson 1 to Lab 1

When generation becomes cheap



Faster production shifts more of our time toward choosing, checking, explaining, and connecting the result to the question.

Understanding is the new bottleneck

Understand to verify

- ▶ Does the code run?
- ▶ Does it match the request?
- ▶ Do the checks pass?
- ▶ Should I keep it?

Understand to participate

- ▶ Why was this approach chosen?
- ▶ Which assumptions are inside it?
- ▶ What question should come next?
- ▶ How should the project change?

Adapted and paraphrased from [Litt \(2026\)](#). Litt argues that understanding supports approval of the current work and active participation in the next project loop.

Cognitive debt

The project can continue while the people responsible for it lose the ability to explain:

- ▶ what the system is supposed to do;
- ▶ why an important decision was made;
- ▶ which assumptions the output carries;
- ▶ how to change the work safely.

This is **cognitive debt**: the work advances faster than shared understanding. The cost appears later, when we need to modify, teach, or repair the project.

Concept adapted from the discussion in [Litt \(2026\)](#), which connects the term to Margaret-Anne Storey and Simon Willison.

Education has two products

The artifact

A script, table, figure, or explanation that works for the current task.

The learner

A person who can ask a new question, notice a mistake, and change the work after the class ends.

Learning remains incomplete when a student keeps an agent-produced artifact without being able to reconstruct its central decision.

What one coding experiment suggests

Shen and Tamkin (2026) randomly assigned AI assistance to 52 developers learning a new Python library.

Immediate product

The AI group finished about two minutes earlier. The difference was not statistically significant.

Later mastery

The AI group scored 50% on the quiz, compared with 67% without AI. The largest gap appeared in debugging.

This small study examines one skill. We use its result as a warning and avoid generalizing it to every learning setting.

Experience still changes the result

People with more task knowledge can usually:

- ▶ give the agent a better plan;
- ▶ notice missing context;
- ▶ request a more relevant check;
- ▶ recover when a plausible answer is wrong.

Observational evidence from coding-agent sessions finds persistent returns to expertise. The authors describe this association and leave its causal interpretation open (Hitzig et al., 2026).

Before, during, and after agent help

Moment	Student action	Useful agent support
Before	Predict the result and state the criterion.	Ask for a plan or a small example.
During	Compare the proposal with your prediction.	Ask why and test a small case.
After	Reconstruct and explain the central decision.	Ask for a quiz or an independent check.

The goal is to keep our understanding close to the speed of production (Shen and Tamkin, 2026; Litt, 2026).

The student standard

Before keeping a substantial block of agent-written code, be ready to state:

1. its purpose;
2. its input and output;
3. one important assumption;
4. one consequence of changing it;
5. one check that could reveal a mistake.

If this is difficult, ask the agent for a smaller example, an explanation with background, or a short quiz. Then try again in your own words.

Use the agent to help you understand

Explainer

Ask for the goal, background, important choices, and a literate walkthrough.

Quiz

Ask five questions that test whether you can reconstruct the central change.

Micro-example

Ask for a small object you can change and observe. Another long explanation may be harder to test.

These suggestions adapt Litt's techniques to a classroom: explanation, retrieval practice, and small environments that the learner can manipulate (Litt, 2026).

Lesson map

Welcome and the question

From a model to an agent

Learning with an agent

Delegating a bounded task

Risk and verifiability

A work order

From Lesson 1 to Lab 1

More autonomy can accumulate more errors

- ▶ A long task requires several decisions to be correct in sequence.
- ▶ One local mistake can influence every later step.
- ▶ Breaking the work into parts makes intermediate results observable.
- ▶ Tests reduce risk only when they check something related to the real question.

Coding agents have improved quickly on long-task benchmarks. Our project still requires its own checks (METR, 2025).

The frontier is irregular

Two tasks that look similar to us may fall on different sides of the system's capability.

Often easier to verify

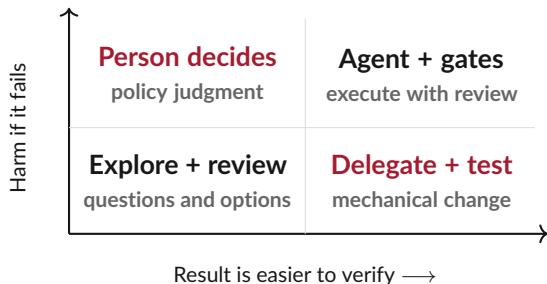
Rename variables, change a format, add a stated assertion, or reproduce a documented transformation.

Often harder to verify

Resolve an ambiguous policy concept, infer missing context, or decide whether an association justifies action.

Confidence gives us no information about where the task was on this “jagged frontier” (Dell’Acqua et al., 2026).

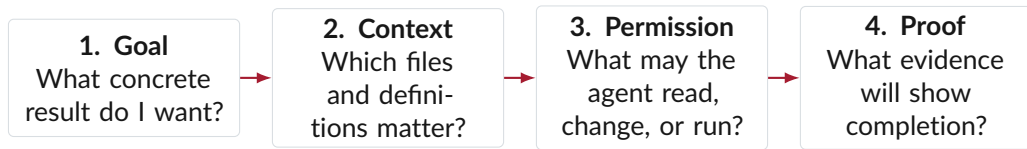
Match autonomy to risk and verifiability



Verifiability is high when a source, test, or independent comparison can expose the error.

Harm includes cost, reversibility, and consequences for people, data, and claims.

Write four things before execution



Describe the proof before delegating the task.

A weak request

Request

Review my project and fix what is wrong.

What is missing?

- ▶ The object is unbounded.
- ▶ Relevant project context is missing.
- ▶ The permission boundary is missing.
- ▶ Evidence is undefined.
- ▶ Completion is undefined.

A bounded, read-only work order

Goal: Explain how a student should start this project.

Context: Read START-HERE.md and the setup guide.

Permission: Read only. Do not edit files or browse the web.

Constraints: Separate facts in the files from your inferences.

Proof: List the steps in order, cite each source file, and end with three checks that the student can run manually.

The request is long because it makes the files, limits, and completion evidence explicit enough for the student to review.

Useful context can change a decision

- ▶ Give the agent the source file directly.
- ▶ Identify the current version and the source of truth.
- ▶ Separate facts, decisions, examples, and open questions.
- ▶ Ask for references to a file, line, table, or URL.
- ▶ Leave out material that cannot change the decision.

Good context is information that can change a decision.

Give permissions according to the task

Reading

Web pages, documentation, code, public data, and metadata.

Reading is a good first step when we are still learning the project.

Action

Writing, running, deleting, publishing, sending, or buying.

Begin with less access. Increase it only when the task needs it.

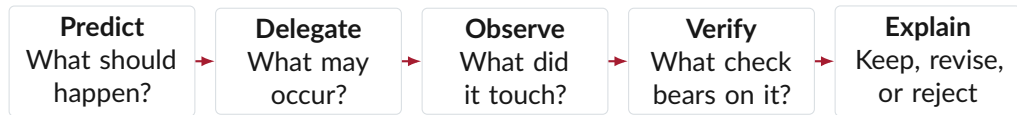
Never provide credentials, API secrets, identifiable student records, or confidential research material to an AI tool.

Define evidence for the product

Product	Minimum evidence
Code	It runs from a clean start and relevant checks pass.
Table or figure	Source, sample, units, and transformations agree.
Explanation	It matches the files and distinguishes fact from inference.
Text	Sources are attributed and no result is invented.

A good check is cheap enough to repeat and strong enough to expose a plausible failure.

Inspect the work behind the answer



“Reject” and “still unresolved” are valid outcomes. Keep a result when the available evidence supports it.

Live read-only demonstration

Watch Codex read the course project without changing it.

Record four things:

1. Which files did it read?
2. Which actions did it request?
3. Which statements came directly from files?
4. Which decision still belongs to us?

The demonstration uses the same project that you will open in Lab 1.

Lesson map

Welcome and the question

From a model to an agent

Learning with an agent

Delegating a bounded task

From Lesson 1 to Lab 1

What we keep

What we learned in Lesson 1

- ▶ Choose the task before choosing the tool.
- ▶ Keep understanding close to the speed of production.
- ▶ Match agent autonomy to risk and verifiability.
- ▶ Define completion evidence before execution.
- ▶ Keep the question, interpretation, and responsibility with the person.

We will reuse this supervision structure during the next lessons as the code changes.

Lab 1 is an installation clinic

The data exercise begins after every laptop passes the readiness gate.

A laptop is ready when the setup check is green. A red result leaves with the exact error, one repair step, and a named person for follow-up.

The readiness gate

Open the Terminal tab in RStudio and run:

```
Rscript code/check_setup.R --codex-confirmed
```

The checker verifies:

- ▶ the course project and the R version;
- ▶ required R packages;
- ▶ Quarto;
- ▶ the frozen teaching data;
- ▶ Codex access to the same project.

Lab 1 route

Minutes	Focus	What we need to see
00-05	Triage	Green, yellow, or red output.
05-15	Project + R	Extracted folder, open project, current R.
15-25	Packages	Install only what the checker reports.
25-35	Quarto	Install, restart RStudio, and rerun.
35-50	Data + Codex	Frozen file and project access.
50-60	Final gate	Green result or an owned repair plan.

Before we leave

Write one sentence for each item:

1. One task I would delegate to a coding agent.
2. One decision I would keep with the person.
3. One type of evidence I will require before I keep generated code.

Bring these answers back when we begin Lesson 2.

References I

Anthropic, “Claude 3.7 Sonnet and Claude Code,” Product announcement February 2025.

Dell’Acqua, Fabrizio, Edward McFowland, Ethan R. Mollick, Hila Lifshitz-Assaf, Katherine C. Kellogg, Saran Rajendran, Lisa Kraye, François Cadelon, and Karim R. Lakhani, “Navigating the Jagged Technological Frontier: Field Experimental Evidence of the Effects of AI on Knowledge Worker Productivity and Quality,” *Organization Science*, 2026, 37 (2), 403–423.

Hitzig, Zoe, Maxim Massenkoff, Eva Lyubich, Shaoyi Zhang, Ryan Heller, and Peter McCrory, “Agentic Coding and Persistent Returns to Expertise,” Anthropic Economic Research June 2026.

Litt, Geoffrey, “Understanding Is the New Bottleneck,” Essay and conference talk July 2026.

METR, “Measuring AI Ability to Complete Long Tasks,” Research report March 2025.

References II

Ng, Andrew, “Skills for Students with AI Goals,” Stanford eCorner interview transcript October 2023.

— , “Building AI Startups: Important Technical AI Trends,” Presentation 2025.

OpenAI, “Codex Use Cases,” OpenAI Developers documentation 2026.

Shen, Judy Hanwen and Alex Tamkin, “How AI Impacts Skill Formation,” arXiv:2601.20245 2026.

Smith, Adam, *An Inquiry into the Nature and Causes of the Wealth of Nations*, London: W. Strahan and T. Cadell, 1776. Book I, Chapter I.